

ATIVIDADE	SOMATIVA - SITUAÇÃO PROBLEMA
CONTEXTO e DESCRIÇÃO	
“Controle de Cobranças e Inadimplência Condominial” A administradora de condomínios onde sua equipe atua atende diversos empreendimentos e precisa controlar a arrecadação mensal de taxas condominiais com agilidade e padronização. Hoje, parte do processo ainda depende de planilhas e conferência manual: emissão de cobranças, acompanhamento de vencimentos, identificação de inadimplência e geração de acordos de parcelamento. A empresa Condom SA, referência no setor, destaca justamente a automatização da arrecadação e do recebimento (boleto, Pix e cartão), além de recursos para gestão de inadimplência e acordos. Com o crescimento do número de condomínios, a administradora solicitou uma primeira versão de um sistema web interno (MVP) com API, para organizar esses dados e preparar uma futura integração com um portal/app para moradores (conceito semelhante ao portal do condômino). Seu desafio é desenvolver uma aplicação com Django e Django REST Framework que simule esse cenário real de trabalho. MISSÃO DO ALUNO Desenvolver uma aplicação web utilizando Django e Django REST Framework para atender a rotina de uma administradora de condomínios, com funcionalidades mínimas de cadastro, cobrança, controle de inadimplência e acordos: Cadastro de Condomínios e Unidades - Cadastrar Condomínios (nome, CNPJ opcional, endereço simplificado) - Cadastrar Unidades (número, bloco opcional, proprietário/morador responsável) vinculadas a um Condomínio - Cada Unidade deve pertencer a exatamente um Condomínio (relacionamento 1:N) Emissão e Controle de Cobranças (Receitas) - Gerar cobranças mensais por Unidade (competência, data de vencimento, valor, status) - Status sugeridos: PENDENTE, PAGO, VENCIDO, CANCELADO - Registrar forma de pagamento quando quitado (ex.: boleto, Pix, cartão), alinhado ao contexto de arrecadação e recebimento amplamente usado em plataformas do setor - Ao pagar uma cobrança após o vencimento, o sistema deve calcular automaticamente multa e/ou juros - A data_pagamento deve ser obrigatória quando status = PAGO Controle de Inadimplência - Identificar cobranças vencidas e classificá-las como inadimplentes após o vencimento - Permitir consulta filtrada de inadimplência por condomínio, unidade e período (competência/vencimento) - Essa etapa simula a necessidade de acompanhar atrasos e organizar ações de cobrança de forma estruturada - Multa: aplicada após o vencimento (2%) - Juros: aplicados por dia de atraso (0,033% ao dia) Acordos de Parcelamento - Criar um Acordo para uma Unidade contendo uma ou mais cobranças vencidas - Definir quantidade de parcelas e gerar automaticamente as parcelas do acordo com datas de vencimento e valores - Disponibilizar endpoint para listar acordos e suas parcelas - A ideia reflete a rotina de “acordos” usada para cobranças inadimplentes em sistemas de gestão condominial Regras adicionais de cobrança - Uma cobrança deve possuir os campos: valor, data_vencimento, status, data_pagamento, multa e juros - Caso a cobrança seja paga após o vencimento, o sistema deve considerar a aplicação de multa e/ou juros - A data_pagamento deve ser obrigatória quando o status for "PAGO" - A unidade deve possuir um campo de status (OCUPADO ou VAGO)	

REQUISITOS TÉCNICOS

- Ambiente virtual configurado
- Projeto Django com app api
- Django REST Framework instalado e configurado
- Modelos com relacionamento ForeignKey:
 - Condomínio → Unidade (1:N)
 - Unidade → Cobrança (1:N)
 - Unidade → Acordo (1:N)
 - Acordo → ParcelaAcordo (1:N)
- Serializers (ModelSerializer) com campos de relacionamento via ID (ex.: unidade_id, condominio_id, acordo_id)
- ViewSets com operações de CRUD
- URLs usando routers do DRF (DefaultRouter)
- Autenticação JWT:
 - Rotas de token (obter e renovar)
 - Regras de acesso: somente autenticado pode criar/editar/excluir; GET pode ser público ou restrito conforme orientação do professor
- Filtros por query parameters com DjangoFilterBackend (exemplos abaixo)

ENDPOINTS MÍNIMOS (padrão sugerido)

- /api/condominios/
- /api/unidades/
- /api/cobrancas/
- /api/acordos/
- /api/parcelas-acordo/

Dashboard financeiro (endpoint inteligente)

- /api/dashboard/

```
{
  "total_condominios": 5,
  "total_unidades": 120,
  "total_cobrancas": 300,
  "total_pagas": 180,
  "total_pendentes": 70,
  "total_vencidas": 50,
  "valor_total_recebido": 85420.00,
  "valor_total_em_aberto": 23110.00,
  "total_acordos": 12
}
```

Inadimplência por condomínio (endpoint inteligente)

- /api/inadimplencia/resumo/

```
[
  {
    "condominio": "Residencial Primavera",
    "qtd_cobrancas_vencidas": 14,
    "valor_total_vencido": 7820.00
  },
  {
    "condominio": "Condomínio Sol Nascente",
    "qtd_cobrancas_vencidas": 8,
    "valor_total_vencido": 3910.00
  }
]
```

Resumo de uma unidade (endpoint inteligente)

- /api/unidades/10/resumo-financeiro/

```
{
  "unidade": 10,
  "responsavel": "Carlos Silva",
  "total_cobrancas": 12,
  "total_pagas": 9,
  "total_vencidas": 2,
  "total_pendentes": 1,
  "valor_em_aberto": 1450.00,
  "possui_acordo": true
}
```

FILTROS (OBRIGATÓRIOS)

1. Cobranças
 - Filtrar por condomínio (via unidade), unidade, status e competência
 - Exemplos:
 - /api/cobrancas/?unidade=10
 - /api/cobrancas/?status=VENCIDO
 - /api/cobrancas/?competencia=2026-02-01
2. Inadimplência (consulta baseada em cobranças vencidas)
 - Pode ser uma rota específica ou filtro em cobranças:
 - /api/cobrancas/?status=VENCIDO&condominio=3
3. Acordos
 - Filtrar por unidade e período de criação:
 - /api/acordos/?unidade=10

DESAFIO PROPOSTO (REGRAS DE NEGÓCIO – MÍNIMO)

- Uma cobrança com status PAGO deve ter data_pagamento preenchida
- Uma cobrança vencida (data_vencimento < hoje e não paga) deve aparecer nas consultas de inadimplência
- Um acordo só pode incluir cobranças da mesma unidade
- Ao criar um acordo, o sistema deve gerar automaticamente as parcelas do acordo

ENTREGÁVEIS

- Código-fonte executando localmente (runserver)
- Banco de dados criado via migrations (makemigrations/migrate)
- API acessível via navegador e/ou Postman/Insomnia nas rotas definidas
- Evidências de testes (prints ou relatório curto) contendo:
 - Criação de condomínio, unidade, cobrança
 - Simulação de cobrança vencida e consulta de inadimplência
 - Criação de acordo com geração de parcelas
 - Requisições autenticadas com JWT (token no header)

DOCUMENTAÇÃO MÍNIMA

Entregar um texto (ou vídeo curto) com:

- Como rodar o projeto
- Quais endpoints existem e exemplos de uso (GET/POST)
- Explicação do relacionamento entre as tabelas (DER simplificado)

HABILIDADES DESENVOLVIDAS

- Modelagem relacional aplicada a um cenário real (condomínios, unidades e cobranças)
- Criação de APIs REST com DRF usando ViewSets e Router
- Autenticação e segurança com JWT
- Filtragem e organização de dados via query parameters
- Implementação de regras simples de negócio (inadimplência e acordos)
- Vivência próxima do contexto profissional de soluções de gestão condominial e arrecadação

REQUISITOS FUNCIONAIS (obrigatório)

Campos obrigatórios da cobrança:

- Valor
- data_vencimento
- status
- data_pagamento (quando pago)
- multa (valor ou percentual)
- juros (valor ou percentual)

Campos adicionais na unidade:

- status da unidade (OCUPADO ou VAGO)

POPULAÇÃO DAS TABELAS

Você deverá utilizar qualquer IA para gerar dados em pelo menos uma planilha, para que se possa popular tabelas desse projeto.

DOCUMENTAÇÃO

Entregar documentação em PDF com as seguintes informações:

- Diagrama e Entidade de Relacionamento
- Requisitos funcionais
- Requisitos não funcionais

Deve entregar também no repositório o arquivo do banco de dados (.sql)

FINALIZAÇÃO

Ao concluir a atividade, o aluno terá construído um sistema funcional alinhado a processos comuns em administradoras de condomínios: cadastro estruturado, controle de cobranças, identificação de inadimplência e formalização de acordos. Além de consolidar fundamentos de Django e APIs REST, o projeto aproxima o estudante da realidade do ambiente corporativo ao reproduzir rotinas e necessidades típicas de plataformas profissionais do setor, preparando-o para evoluir soluções com segurança, organização e foco em regras de negócio.

CRITÉRIOS CRÍTICOS (50 pontos — mínimo para funcionamento)

Nº	Critério	Nota
1.	Estrutura do projeto Django + DRF configurado corretamente	3
2.	Modelos criados corretamente	3
3.	Relacionamentos entre tabelas funcionando	3
4.	Migrações aplicadas e banco funcionando	2
5.	Serializers básicos funcionando	4
6.	Endpoints de cobranças, inadimplências e acordos	6
7.	Autenticação JWT funcionando (/token/)	6
8.	Proteção básica de endpoints (IsAuthenticated)	4
9.	Testes básicos via Postman/Insomnia	2
10.	Implementação correta dos campos de cobrança (multa, juros, data_pagamento)	3
11.	Cálculo automático de multa e juros para cobranças vencidas	2
12.	Código limpo e padronizado (views limpas, separação de lógica)	2
13.	Domínio do projeto (pergunta oral)	10
14.	Total	50

CRITÉRIOS DESEJÁVEIS (50 pontos — nível avançado)

Nº	Critério	Nota
15.	CRUD completo (todos os modelos) bem estruturado	4
16.	Filtros avançados (pelo menos 3 filtros)	4
17.	Endpoint de medições recentes (lógica correta)	3
18.	Validações de regra de negócio	2
19.	Tratamento de erros HTTP (400, 401, 404 padronizados)	1
20.	Distinção de níveis de usuários (Admin pode tudo e User só visualiza)	7
21.	Configuração e funcionamento do banco em MySQL	3
22.	Endpoint inteligente com cálculo ou consolidação de dados funcionando corretamente	2
23.	Endpoint de dashboard/resumo financeiro com regra de negócio aplicada	2
24.	Front: proteção de rotas (sem token não acessa)	2
25.	Front: navegação funcional (login → listagem → detalhe)	1
26.	Front: consumo de múltiplos endpoints	1
27.	Front: envio de dados (POST funcionando)	1
28.	Front: logout funcional	1
29.	Versionamento organizado (GitHub correto)	1
30.	Documentação	5
31.	Front: armazenamento do token	1
32.	Front: consumo de pelo menos 1 endpoint (listagem)	1
33.	CRUD básico funcionando	8
	Total	50